An In-Network Timing Framework for Concealing the Response-Time Fingerprint of a Separate-Acknowledgment DNP3 Device

Anonymous submission

I. Introduction

A passive observer on a control-system network learns a great deal without decrypting anything. The sizes, the rhythms, and above all the response times of a device's traffic form a fingerprint that reveals its type, its vendor, and its role, and an attacker uses that fingerprint as reconnaissance, to decide which hosts matter and how they will behave [1]–[3]. The concern is sharpest in industrial control systems, where the field devices are long lived, their roles are stable, and the very regularity of their polling exposes them. Prior work has fingerprinted supervisory and field equipment directly from its traffic [4], [5], and one of the most reliable features is timing. A DNP3 outstation answers a master's poll first with a transport acknowledgment and then, after a short and characteristic delay, with the response itself. That delay, the cross-layer response time (CLRT), separates control-system devices by vendor and model [5], and because DNP3 is plaintext an on-path observer reads it straight from routine polling.

The natural defense is traffic obfuscation: add cover packets, pad sizes, and reshape flows until the observer can no longer tell one source from another [6]–[9]. That body of work, however, was built for a setting industrial traffic does not share, and the mismatch is not a detail to tune away but the crux of the problem. Internet obfuscation assumes the traffic is encrypted or otherwise opaque, so the shaper may freely inject packets and rewrite flows, because the observer cannot distinguish a real exchange from a fabricated one [6], [8]; even then, the weaker of these defenses have been broken by stronger analysis [10], [11]. Legacy industrial traffic violates every one of those assumptions:

- It is plaintext and checksummed. Each DNP3 link block is readable and CRC-protected [12], [13], so an injected dummy or an edited byte is exposed by inspection rather than hidden by encryption. Cover traffic and byte rewriting, the workhorses of Internet obfuscation, are unavailable.
- Its endpoints are fixed. The outstation runs vendor firmware and DNP3 is a deployed standard [12], [14]; the defense cannot enlist the device or the protocol to reshape their own traffic.
- Its correctness is load-bearing. Every poll must receive its true response, in order, within DNP3's timing, retransmission, and quality-of-service limits, or the control loop breaks.
- Its fingerprint is temporal. The leak is in the response timing, which size- and volume-oriented defenses leave untouched.

What remains is a narrower and unfamiliar problem. A defense here must sit in the network, because the endpoints are immovable; it must preserve every original byte, because the traffic is plaintext and checksummed; it must keep the exchange correct; and it must act on the timing itself, the one thing that leaks. No existing obfuscation technique satisfies all four requirements together, and closing that specific gap, rather than retuning an Internet defense, is what this paper does.

We meet the four requirements with an in-network defense on a single programmable switch placed between the master and the device. The switch never alters a packet's contents; it changes only when the device's acknowledgment and response become externally visible. It holds the original acknowledgment and response in its queues, issues small internal marker packets that recirculate to reserve their places, and lets the traffic manager release the held packets on a schedule the control plane sets [15]–[17]. The design deliberately accommodates a hard limit of data-plane hardware: a switch cannot pause an enqueued packet and summon it back at an arbitrary later time under software control. Gates on the acknowledgment and on the response compose into a small family of timing transformations, from releasing the acknowledgment only once its matching response is ready, through holding either to a fixed deadline, to holding both, all as run-time modes of one mechanism. We treat the switch as a trusted observation and control point on the plaintext path, and the adversary as a passive observer on the master-facing segment, where only the shaped timing is visible.

Contributions. Each contribution is bounded by the evidence we report.

- We articulate why the standard traffic-obfuscation toolkit does not apply to legacy ICS traffic, and we cast the cross-layer response time, which Formby et al. [5] identified as a device-fingerprinting feature, as a shaping target reachable in the network under four constraints, in-network, byte-preserving, correctness-preserving, and timing-directed, that set it apart from the size- and encryption-oriented defenses.
- We design one queue-resident timing mechanism whose event, acknowledgment-deadline, response-deadline, and dual-deadline behaviors are configurations rather than separate systems. Because a separate-acknowledgment device emits its acknowledgment before its response, the response obligation must outlive the acknowledgment's release; we show how the transaction state is kept alive across that release so the later response is still shaped rather than forwarded in the clear.
- We reconcile the DNP3, TCP, and Tofino constraints the design must honor: matching each transaction by its DNP3 generation code on plaintext traffic, keeping the internal markers off the observable wire, preserving the original bytes, and fitting the logic within a single Tofino-1 ingress pipeline.
- We implement the mechanism on an Intel Tofino-1 and evaluate it on silicon against a physical SEL-751 relay. On the corrected implementation the response-deadline and dual-deadline modes drive the measured response time from its broad native distribution to a fixed value and shape every response, a result verified by a fail-closed measurement pipeline, reproduced across two campaigns, and quantified as a sharp collapse of the timing observable. We report these results, and their limits, for one relay, for READ polling, and for the response-time observable; we do not claim cross-vendor indistinguishability, size concealment, or a controlled failure-injection suite, which we leave to future work.

References

[1] T. Kohno, A. Broido, and K. C. Claffy, "Remote physical device fingerprinting," IEEE Transactions on Dependable and Secure Computing, vol. 2, no. 2, pp. 93–108, 2005, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [13].

[2] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A technique for physical device and device type fingerprinting," IEEE Transactions on Dependable and Secure Computing, 2014, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [18].

[3] G. Shu and D. Lee, "Network protocol system fingerprinting – a formal approach," in Proceedings IEEE INFOCOM 2006, 2006, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [19].

[4] S. Jeon, J.-H. Yun, S. Choi, and W.-N. Kim, "Passive fingerprinting of SCADA in critical infrastructure network without deep packet inspection," 2016, verified via arXiv MCP (export_citations). [Online]. Available: https://arxiv.org/abs/1608.07679

[5] D. Formby, P. Srinivasan, A. Leonard, J. Rogers, and R. Beyah, "Who's in control of your control system? device fingerprinting for cyber-physical systems," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2016, anchor paper, read in full. The CLRT device-fingerprinting threat model.

[6] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic morphing: An efficient defense against statistical traffic analysis," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2009, metadata verified from Ditto (NDSS 2022) bibliography, ref. [100].

[7] X. Cai, R. Nithyanand, and R. Johnson, "CS-BuFLO: A congestion sensitive website fingerprinting defense," in Proceedings of the 13th Workshop on Privacy in the Electronic Society (WPES), 2014, venue corroborated by Ditto (NDSS 2022) ref. [36]; metadata via arXiv MCP (1401.6022, "New Approaches to Website Fingerprinting Defenses").

[8] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an efficient website fingerprinting defense," in Computer Security – ESORICS 2016, 2016, wTF-PAD. Venue corroborated by Ditto (NDSS 2022) ref. [67]; metadata via arXiv MCP (1512.00524).

[9] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the smart home private with smart(er) IoT traffic shaping," Proceedings on Privacy Enhancing Technologies (PoPETs), 2019, stochastic Traffic Padding (STP). Venue corroborated by Ditto (NDSS 2022) ref. [26]; metadata via arXiv MCP (1812.00955).

[10] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-boo, i still see you: Why efficient traffic analysis countermeasures fail," in 2012 IEEE Symposium on Security and Privacy (S&P), 2012, buFLO. Metadata verified from Ditto (NDSS 2022) bibliography, ref. [50].

[11] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2018, venue corroborated by Ditto (NDSS 2022) ref. [92]; metadata via arXiv MCP (1801.02265).

[12] S. East, J. Butts, M. Papa, and S. Shenoi, "A taxonomy of attacks on the DNP3 protocol," in Critical Infrastructure Protection III (IFIP AICT, vol. 311). Springer, 2009, pp. 67–81, web-verified (Springer / IFIP CIP III).

[13] I. N. Fovino, A. Carcano, T. D. L. Murel, A. Trombetta, and M. Masera, "Modbus/DNP3 state-based intrusion detection system," in 2010 24th IEEE International Conference on Advanced Information Networking and Applications (AINA), 2010, pp. 729–736, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [10].

[14] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks against process control systems: Risk assessment, detection, and response," in Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS), 2011, pp. 355–366, metadata verified from Formby et al. (NDSS 2016) bibliography, ref. [6].

[15] L. Wang, H. Kim, P. Mittal, and J. Rexford, "Programmable in-network obfuscation of traffic," 2020, verified via arXiv MCP (get_abstract). PINOT; Tofino prototype. [Online]. Available: https://arxiv.org/abs/2006.00097

[16] R. Meier, V. Lenders, and L. Vanbever, "ditto: WAN traffic obfuscation at line rate," in Proceedings of the Network and Distributed System Security Symposium (NDSS), 2022, anchor paper, read in full. Closest in-network line-rate obfuscation prior work.

[17] E. F. Kfoury, J. Crichigno, and E. Bou-Harb, "An exhaustive survey on P4 programmable data plane switches: Taxonomy, applications, challenges, and future trends," 2021, verified via arXiv MCP (export_citations). Also in IEEE Access, 2021. [Online]. Available: https://arxiv.org/abs/2102.00643